PORTER: A Plan-Committed Router for Multi-Core Distributed Quantum Computers

Fuchao Sheng, Yongjian Xie, Yongming Li, Sanjiang Li *

***Abstract*—A distributed quantum computer runs one circuit across several small cores joined by links that distribute entanglement on demand; under the usual cost model the compiler minimises the Einstein–Podolsky–Rosen (EPR) pairs spent on gates that straddle cores. Some existing routers fold into an inter-core action's score the preparation the current layout demands for it, so its apparent value depends on the layout the compiler happens to occupy. We present PORTER, which instead ranks complete inter-core *plans* — which primitive fires, which gates it resolves, which readiness conditions it needs — on the gate progress each buys, letting preparation cost only break ties, then commits to one and prepares it. It supports the three inter-core primitives studied under this cost model, including cat-entangler packets that amortise one pair over a run of remote gates sharing a control, and adds capacity-aware relief, a deterministic initial layout, and a teleport-lookahead horizon chosen per circuit from a fixed four-value portfolio. On MQT-Bench suites at 25, 64 and 80 logical qubits PORTER uses 59.7%, 44.9% and 68.8% fewer EPR pairs in geometric mean than published TeleSABRE at four routes to its three, over the instances both complete, and completes all twenty-four instances to TeleSABRE's seventeen. Against dSABRE it is competitive on those suites — 0.99× its pooled EPR at 1.20× its compile time, each under its own protocol — and pulls ahead as the machine grows, using 0.69× its pairs on a 100–360-qubit QFT and QPEexact scale-up series. The two prove complementary rather than ranked, and taking the better of the two per instance uses 9.4% fewer pairs than dSABRE alone. Controlled experiments separate the contributions of capacity relief, the set of primitives, the initial layout and the horizon; relief earns its place on completion rather than cost, five of nine 80-qubit circuits failing outright without it. The implementation, harness and raw results are public.**

***Index Terms*—distributed quantum computing, quantum circuit compilation, multi-core quantum architectures, gate teleportation, cat-entangler packets, plan-oriented routing, entanglement cost**

I. Introduction

A distributed quantum computer (DQC) runs one circuit across several small cores joined by links that distribute entanglement on demand [1], [2]. Any two-qubit gate whose operands sit in different cores must be paid for with Einstein–Podolsky–Rosen (EPR) pairs, and distributing a pair is far slower and less reliable than any operation inside a core [3], [4]. Pairs consumed is therefore the metric we minimise (Section II-B).

A pair can be spent three ways. Teleporting a *state* (TeleData) relocates a logical qubit, making every later gate it meets in its new core local [5]. Teleporting a *gate* (TeleGate) executes one straddling controlled gate through a cat-entangler, with neither operand moving [6], [7]. Holding that resource open across a run of straddling gates sharing a control costs one pair for the whole run — a *packet*, following AutoComm's amortisation [8]. Which is cheapest depends on the circuit structure where it is applied, so none dominates.

The choice must be made before its price is known. Each alternative becomes executable only after its own preparation — the right operand beside the right communication port, that port free, the destination holding a spare slot — and a compiler performs that work only for the alternative it selects. Scoring an action together with the preparation it currently needs confuses what remote work is worth doing with how far the layout is from doing it: candidates are ranked by the layout the compiler happens to occupy rather than by the progress they buy, and a router that reopens the choice mid-preparation can be left carrying SWAPs towards an action it abandons.

PORTER — a *Plan-Oriented Router for Teleportation-Enabled aRchitectures* — separates the two questions. Its unit of choice is a *plan*: a complete specification of one inter-core action and the conditions it needs, omitting the SWAP sequence that would establish them. Plans are compared on the gate progress they buy, one is committed, and only then are preparation SWAPs generated for it. Around this loop PORTER adds two-tier capacity relief, which manufactures free slots when every candidate is blocked, and a single deterministic initial layout in place of the best-of-N search its predecessors run.

Results. We evaluate on MQT-Bench [9] suites at 25, 64 and 80 logical qubits, a 100–360-qubit scale-up series, and two networks of IBM heavy-hex cores — forty-eight circuit–architecture instances. Against TeleSABRE [10], the strongest published router we identified under this cost model, PORTER lowers geometric-mean EPR consumption by 59.7%, 44.9% and 68.8% on the three suites at four routes to its three, over the instances both complete, and it completes all twenty-four — including the seven TeleSABRE does not converge on, among them an 80-qubit, 23,381-CX circuit.

The second comparison is with dSABRE [11], concurrent work on the same problem and the closest point of reference we have. PORTER is competitive on the three benchmark suites, spending 0.99× its pooled EPR for 1.20× its compile time under each compiler's own protocol, and pulls ahead as the machine grows: on the scale-up series it uses 0.69× dSABRE's pairs, at 0.52× its compile cost, and wins every instance.

A pooled figure that close invites the question of what a

F. Sheng, Y. Xie, and Y. Li are with the School of Mathematics and Statistics, Shaanxi Normal University, Xi'an, China; S. Li is with the Centre for Quantum Software and Information, University of Technology Sydney, Australia (e-mail: sanjiang.li@uts.edu.au).

*ORCID: 0000-0002-3332-2546.

second compiler adds, and the per-instance record answers it: neither wins more than eleven of the twenty-four, and taking the better of the two per instance uses 9.4% fewer EPR pairs than DSABRE alone — 8.2% with the two cheapest circuit families set aside. A design that merely re-derived DSABRE's decisions would leave such a portfolio nothing to choose between. The two are close relatives and share four design decisions, set out in Section VII so neither is credited with the other's; what the portfolio measures is the part that is not shared.

Problem. Given a circuit, a multi-core machine and an initial layout, emit a legal operation stream realising every gate while minimising EPR pairs consumed. Section II-B states this formally, with the cost model it assumes and what that excludes.

Contributions. We make three.

(i) *The* PORTER *compiler*: a plan-oriented remote stage that scores a complete inter-core action before any preparation SWAP for it exists and commits before preparing (Section III-D), a capacity discipline that treats free slots as a legality condition and manufactures them by a two-tier relief search (Section III-G), and a deterministic initial layout replacing the best-of-N search its predecessors run (Section III-A).

(ii) *A lookahead-horizon portfolio, selected per circuit.* Every SABRE-descended router fixes this parameter by constant, and no tested constant is best across our benchmark — the best value spans a factor of thirty-two. PORTER routes every circuit at the same four horizons and keeps the lowest-EPR result. Nothing is conditioned on the circuit, so there is no threshold to generalise, and because the configured horizon is one of the four and wins ties, the portfolio cannot return a worse route than routing once. It is what carries PORTER past DSABRE at scale (Section IV-E1).

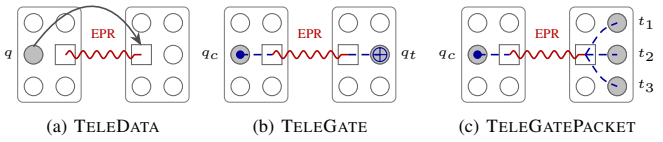
(iii) *A controlled study over forty-eight instances*, under native, shared-layout and cross-layout protocols, with ablations measuring what each mechanism contributes — including a negative finding, which Section VI interprets. Every route is certified by a separately implemented replay verifier, and the harness reproducing every table is released.

Section IV-A states the four questions the evaluation answers and ties each mechanism to the experiment that assesses it. The implementation, the replay verifier, the comparison harness and the raw per-circuit results are available at https://github.com/ebony72/porter.

II. Model, Primitives, and Problem Formulation

A. Quantum Circuits and Routing

A circuit acts on n logical qubits in the basis $\{\texttt{id}, \texttt{rz}, \texttt{sx}, \texttt{x}, \texttt{cx}\}$ of MQT-Bench's Qiskit-opt3-transpiled circuits [9] — one-qubit gates plus CX — and is a directed acyclic graph with a dependency edge between consecutive gates on each logical qubit. The *front* is the set of gates with no unexecuted predecessor; only front gates can execute or be routed toward execution.



(a) TeleData (b) TeleGate (c) TeleGatePacket

Fig. 1. The three communication primitives; each consumes one EPR pair between the two communication ports (squares). (a) TeleData moves qubit q (which must sit adjacent to its free port) onto the free destination port. (b) TeleGate executes one remote CX through a cat-entangler [7] (Appendix E): the target-side port acts as a proxy for the control q_c; neither qubit moves. (c) TeleGatePacket brackets an ordered run of remote CX gates sharing the control q_c between one cat-entangler/disentangler pair, amortising a single EPR pair over the whole run; each target is routed next to the port in turn while the ports stay reserved.

Physical qubits are partitioned into *cores*. Each core has an intra-core coupling graph, and pairs of cores may be joined by an inter-core link whose endpoints are *communication ports*. Every device here carries one link per adjacent pair and one port per neighbour, distinct across neighbours, so an adjacent pair fixes its port pair and a plan has no port to choose. A *layout* maps every logical qubit to a physical position. A two-qubit gate is *local* if both operands share a core and *remote* otherwise: a local gate that is not yet adjacent needs intra-core SWAPs, a remote gate needs an inter-core primitive. We write $\kappa_\ell(q)$ for the core holding q under layout ℓ, $d_{\mathrm{L}}(c, p, p')$ for intra-core distance and $d_{\mathrm{C}}(c, c')$ for core-graph distance.

B. Objective, Scope, and Assumptions

Objective. Given a circuit DAG C on n logical qubits, a machine $\mathcal{A}$ of K cores with intra-core coupling graphs, inter-core links and designated communication ports, and an initial layout ℓ_0, produce a legal operation stream that realises every gate of C while minimising the number of EPR pairs consumed. Intra-core SWAP count, circuit depth and compile time are recorded but are not part of the objective and are never traded against EPR pairs by any rule in this paper.

Cost model. One EPR pair is consumed per TeleData, per TeleGate and per TeleGatePacket — one per packet, however many remote CX gates it amortises — charged the same whichever link carries it; a multi-hop relocation is a sequence of single-hop TeleData actions, charged once per hop. This is the model TeleSABRE [10] and DSABRE [11] use, which is what makes the three compilers comparable in one unit. Communication ports are ordinary physical qubits, occupied for an action's duration and free otherwise; no core is assumed to have dedicated communication memory. What the model leaves out — link contention, entanglement latency and fidelity, purification, classical communication — is outside it, and every conclusion here is relative to that model. Appendix K collects this and the evaluation's other assumptions in one place.

C. Communication Primitives

Three inter-core primitives are in play, sketched in Figure 1; each consumes exactly one EPR pair.

State teleportation (TeleData). Consumes one EPR pair to move a logical qubit from a communication port in one core

to a free port in another, after which every later gate that qubit meets in its new core is local. Its value is therefore *latent*: it is paid once and repaid over however many future gates the relocation makes local.

Gate teleportation (TELEGATE). Executes a single remote CX with neither operand moving. One EPR pair entangles the two communication ports; a *cat-entangler* leaves (q_c, e_B) in the cat state $\alpha|00\rangle + \beta|11\rangle$, so every CX controlled by the remote port e_B acts as if controlled by q_c, and a *cat-disentangler* then restores q_c and frees both ports (Appendix E gives both constructions). Its value is entirely *direct*: exactly one gate, nothing left behind.

Same-control packets (TELEGATEPACKET). When several remote CX gates share the same control and their targets lie in one core, the run can be bracketed between a *single* cat-entangler and disentangler: the ports stay reserved, each target is routed beside e_B in turn, and the whole run costs one EPR pair instead of one each. This is the amortisation idea of AutoComm [8], specialised to the shared-control case and applied inside a routing loop rather than as a pre-pass.

The three differ in the respect that matters for scoring: how many gates one pair is spread over. TELEDATA amortises over an unknown future, TELEGATE over exactly one gate, TELEGATEPACKET over a run whose length the compiler must discover. Section III-E makes that a direct/latent/lookahead decomposition.

III. THE PORTER COMPILER

Given a circuit and an initial layout ℓ_0 (which PORTER builds itself, Section III-A), PORTER repeats the following loop (Algorithm 1) until every gate has been emitted, a configurable iteration budget is exhausted, or a configurable number of iterations passes with no gate retired — a no-progress guard that abandons a stalled route without waiting for the budget. Algorithm 1 charts the routing pass; lines 11–19 are the plan-first remote stage that distinguishes PORTER from compilers deciding at single-action granularity, and Appendix F walks through them on a concrete front where they change the outcome. This section specifies one routing pass, at a fixed teleport-lookahead horizon L_{tp}. PORTER's own protocol routes at four candidate horizons and keeps the lowest-EPR result; because that invokes this loop repeatedly rather than changing it, it is defined with the other operating protocols in Section IV-E1.

Algorithm 1 PORTER `compile`(C, ℓ_0)

1: $\ell \leftarrow \ell_0$; initialise DAG active and front sets from C
2: **while** DAG has active gates **do**
3: $F \leftarrow$ DAG front
4: classify F into singles, direct-local CX, local-CX-needing-SWAP, adjacent-remote CX, non-adjacent-remote CX *(Sec. III-B)*
5: **if** singles or direct-local CX present **then**
6: execute them all; remove from DAG; **continue**
7: **end if**
8: **if** local-CX-needing-SWAP present **then**
9: generate candidate intra-core SWAPs, apply the best-scoring one (Eq. 1); **continue** *(Sec. III-C)*
10: **end if**
11: generate normal plans $\mathcal{P}$, blocked plans $\mathcal{B}$ *(Sec. III-D)*
12: score every plan in $\mathcal{P} \cup \mathcal{B}$ *(Sec. III-E)*
13: $\mathsf{p}^* \leftarrow$ first plan of $\mathcal{P}$ in selection-key order (`None` iff $\mathcal{P} = \emptyset$)
14: **if** p^* is `None` **then**
15: $\mathsf{p}^* \leftarrow$ best tail-relief plan seeded from $\mathcal{B}$ *(Sec. III-G)*
16: **else if** $\mathcal{B} \neq \emptyset$ and $\max_{\mathsf{b}\in\mathcal{B}} \text{total}(\mathsf{b}) - \text{total}(\mathsf{p}^*) > m_{\text{assist}}$ **then**
17: $\mathsf{b}^* \leftarrow \arg\max_{\mathsf{b}\in\mathcal{B}} \text{total}(\mathsf{b})$
18: $\mathsf{p}^* \leftarrow$ best relief plan for b^*'s blocked core *(Sec. III-G)*
19: **end if**
20: **if** p^* is `None` **then**
21: **report deadlock; abort**
22: **end if**
23: apply improving preparation SWAPs until p^* is ready *(Sec. III-D)*
24: execute p^*
25: **end while**
26: **return** emitted operations, final layout, stats

A. Initial Layout

PORTER routes from any initial layout the machine has spare capacity for (Section III-H, which also legalises a supplied layout whose cores are full), and the shared- and cross-layout experiments of Section V-B run it on layouts built by other compilers. Left to itself it builds ℓ_0 with one deterministic heuristic — no external partitioner, no best-of-N search — giving every core at least $f_{\min}$=2 free positions.

That floor is a requirement rather than a tuned parameter, and DSABRE [11] independently arrives at the same value. A core with no free position cannot vacate an occupied communication port by local SWAPs, so nothing can reach it; a core with one free position cannot accept a TELEDATA, since the arriving qubit would fill it. Two is the smallest reserve under which a core can both receive a teleport and still make room afterwards. $f_{\min}$ accordingly plays three roles, which we keep apart: this *initial* condition; the *admission threshold* a destination must meet *before* a normal TELEDATA (Section III-D); and the threshold defining a *reservoir* for relief (Section III-G). It is not an invariant — a core admitted at exactly $f_{\min}$ drops to one free, and cores do reach one free position during routing.

Temporal locality-greedy. Three steps. *(i) Weighting*: each CX gate $g = (q_i, q_j)$ contributes $\gamma^{\text{asap}(g)}$ to the interaction weight $w(q_i, q_j)$, where $\text{asap}(g)$ is g's as-soon-as-possible layer in the CX-only DAG and γ=0.99, so early interactions weigh more than late ones while repeated interactions accumulate. *(ii) Core assignment*: logical qubits are visited in the order they first appear as a CX operand and each is placed in the core holding its heaviest already-placed neighbour when that core has headroom above $f_{\min}$, falling back to the core with the most remaining reserved capacity. *(iii) Physical placement*:

within each core, logicals are laid onto positions by descending interaction weight onto ascending intra-core centrality, so the most-connected qubits take the most central slots.

The steps are complementary. The discount co-locates the pairs that must interact soonest; first-appearance order handles chain-structured circuits, where a balanced-first-layer rule would scatter the chain across cores; and the centrality step keeps hub qubits cheap to reach. Its one weakness is that the $f_{\min}$ reservation is uniform across cores, which suits a grid or a ring but not a core graph with a hub (Section V-E).

B. Front Classification

Every front gate is classified under the current layout ℓ. 1Q gates are always directly executable; a CX $g = (q_i, q_j)$ with both operands in core c is *direct-local* if $d_{\mathrm{L}}(c, \ell(q_i), \ell(q_j)) = 1$ and *local-needing-SWAP* otherwise; a CX whose operands lie in different cores is *adjacent-remote* if those cores are core-graph neighbours and *non-adjacent-remote* otherwise. Directly executable gates drain first and completely, intra-core SWAPs are exhausted next, and remote-stage decisions are taken only once no front gate is intra-core.

C. Intra-Core SWAP Scoring

This stage is SABRE [12]. For a local-needing-SWAP gate set F_{loc}, candidate SWAP edges are the coupling-graph neighbours of every physical position F_{loc} touches, and each is scored under its post-SWAP layout ℓ_a by

$$J(a) = \bar{d}_{\mathcal{F}}(a) + \lambda_{\mathrm{loc}}\, \bar{d}_{\mathcal{E}}(a), \qquad \lambda_{\mathrm{loc}} = 0.8, \tag{1}$$

the mean intra-core distance $d_{\mathrm{L}}(c, \ell_a(q_i), \ell_a(q_j))$ over F_{loc}, plus the same mean over an extended set of up to N_{loc}=20 local CX gates from the next L_{loc}=12 DAG layers. The lowest J is applied; ties break on raw front-layer distance, then on a constant-seeded generator. Note the sign convention differs between the two stages: local candidates *minimise* distance, whereas remote plans *maximise* gain (Section III-E).

D. Plan-First Remote Stage

PORTER manipulates remote work as *plans*. A plan is a complete, committable specification of one inter-core action: which primitive fires, which gate or same-control run it resolves, its direction, and consequently its *readiness conditions* — which logical endpoint must sit adjacent to which communication port, and which ports must be free. What it deliberately leaves unspecified is the SWAP sequence that establishes those conditions. Alternatives are compared at the level of *what* remote work to do next, each scored as if its preconditions were met; only the committed plan is handed to the machinery that works out *how*.

Formally, a plan is a tuple

$$\mathsf{p} \;=\; \big(\pi,\; g,\; G(\mathsf{p}),\; q,\; c_{\mathrm{src}},\; c_{\mathrm{dst}},\; \mathrm{mode},\; \Pi(\mathsf{p})\big),$$

with $\pi \in \{\textsc{TeleData}, \textsc{TeleGate}, \textsc{TeleGatePacket}\}$ the primitive, g the front CX that seeds it, q the qubit whose state crosses the link, c_{src} the core holding q, c_{dst} the core at the other end, mode the generating rule, and $\Pi(\mathsf{p})$ the inter-core link it reserves. Every field is defined for every primitive. For a TeleData, q is the qubit that moves; for the TeleGate family it is the *control*, whose state the cat entangler copies to the target side and which every gate of a packet shares by construction. mode records which of three rules generated a TeleData: *localise*, when the seed CX's endpoints sit in adjacent cores and moving one into the other's makes the gate local; *reduce distance*, when they sit further apart and one moves a hop towards the other; and *relieve capacity*, when the seed is not a front CX but a plan a capacity floor blocked, and the eviction goes to a reservoir (Section III-G). The TeleGate family has one rule — execute the straddling gates in place — so mode is what separates TeleData plans sharing a primitive and a core pair. Both cores are defined for every primitive, and c_{dst} is always a neighbour of c_{src}: the TeleGate family is generated only for a core-adjacent CX, a TeleData only for a one-hop move, and the tail-relief search emits the last hop of a chain rather than the chain, so every plan acts across a single link. The seed and the moving qubit are what individuate a TeleData: its $G(\mathsf{p})$ is empty, so without them two plans moving different qubits between the same pair of cores would be the same tuple. On the single-link interconnect assumed here $\Pi(\mathsf{p})$ follows from the two cores, and it becomes an independent choice exactly when a core pair is joined by more than one link. Every field is static: a plan is fixed once generated, so commitment has a fixed object. Readiness is not a field but a predicate derived from a plan and a layout, $\mathrm{ready}(\mathsf{p}, \ell)$, holding when p can fire under ℓ; preparation changes ℓ and so changes the predicate, never the plan. $G(\mathsf{p})$ is the set of gates it *discharges*: for a TeleGate the single front CX; for a TeleGatePacket the collected same-control run (Section III-F); for a TeleData the empty set, since moving a qubit executes no gate — its value lies in the gates the relocation makes local, which the scorer estimates (Section III-E).

Three quantities are kept distinct: the plan's *utility*, evaluated under the hypothesis that $\mathrm{ready}(\mathsf{p}, \ell)$ holds whether or not it does; its *readiness cost*, the local work still needed, used only to break ties; and the *preparation policy* that chooses those local SWAPs after p is committed. Two design decisions follow: utility never includes readiness cost, and once a plan is committed PORTER works only on that plan — it generates no other plan and scores no alternative — until the plan fires, which Section III-D shows it always does.

The second is the *commitment rule*, and we adopt it on structural grounds rather than measured ones. A preparation SWAP has no meaning except relative to the plan it prepares, so a router that reopens the choice part-way through preparation can leave the layout carrying moves made towards an action it then abandons; committing makes preparation and the action it serves one unit. It also fixes the granularity at which the plan set is derived — once per committed action rather than once per router iteration. What it does *not* come with is an experiment: no result in Section V isolates either the commitment rule or the plan abstraction beneath it, so no reduction reported there is attributed to them; separating their share would need a hop-granular candidate set, which is a different router.

The candidate sets. When the front holds only remote CX gates, PORTER builds two sets. **Normal plans** $\mathcal{P}$ are the actions that could fire once prepared. For each adjacent-remote CX they hold a TELEGATE; a TELEGATEPACKET if a same-control run of ≥ 2 gates can be collected; and, for either endpoint, a TELEDATA into the partner's core when that core has $f_{\min}$ free positions. A non-adjacent-remote CX cannot be discharged in one action, so it contributes only TELEDATA hops that move one endpoint to a neighbouring core *strictly closer* to the other endpoint's core, again subject to $f_{\min}$.

Blocked plans $\mathcal{B}$ are the same TELEDATA candidates whose destination fails the $f_{\min}$ test. PORTER never commits one — executing it would drive a core below the floor. It scores them anyway, as a measurement of what capacity is currently costing: $\mathcal{B}$ names the cores that are saturated and says how much gain is being forgone there, which is what the relief tiers of Section III-G act on.

Selection. Every plan in both sets is scored before any preparation SWAP for it exists; Section III-E gives the score and the ordering key (2) it induces. PORTER then walks $\mathcal{P}$ in that order and commits to the first one. It needs no admissibility test: a plan is *ready* when its endpoints already sit beside the ports it needs and those ports are free, so it can fire immediately, and every plan that is not ready is *preparable* — some local SWAP moves the layout closer to readiness — which Section III-D shows always holds. Readiness is therefore not a selection criterion. It enters the key only through prep, which is zero exactly when a plan is ready, and it is what tells commitment when to stop preparing and fire.

Two things can override the choice, and both are driven by the single best-scoring member of $\mathcal{B}$ alone — call it b^*; the rest of $\mathcal{B}$ enters no comparison. If a plan was selected but b^* out-scores it by more than m_{assist}=5.0, PORTER drops the selected plan and relieves the core blocking b^* instead. If $\mathcal{P}$ is empty, it relieves $\mathcal{B}$'s blocked destinations in ascending hop order, taking the highest-scoring blocked plan among those at the smallest hop count as the seed. In both cases what is committed is a relief TELEDATA generated for the purpose, never b^* itself. Having committed, PORTER applies only that plan's preparation SWAPs until it is ready, and then executes it. That loop has one exit, not several: Section III-D shows a committed plan can always be prepared, so selection needs no ready-or-preparable filter and commitment needs no exclusion list for plans it gave up on.

Preparation SWAPs. A committed plan has exactly two readiness requirements, one per side: source and destination for TELEDATA, control and target for TELEGATE and TELEGATEPACKET, whose target side is routed at execution time (Section III-F). For each unmet requirement, candidate SWAPs are the edges incident to the plan's logical endpoint, or to a nearest free position tied for minimum distance to the relevant port while that port stays occupied. Each is scored by the post-SWAP readiness cost, and at most one improving SWAP per side is applied per pass.

Preparation always completes. A committed plan is never abandoned, because an unsatisfied side never has an empty candidate set. Routing never fills a core (Section III-H), and $f_{\min} = 2$ is exactly the value making that inductive: a TELEDATA needs two free positions at its destination and consumes one. With a free position in every core and cores internally connected, an unsatisfied side carrying a logical endpoint always admits a SWAP, since that endpoint's own position is occupied and every intra-core edge incident to it is legal. The one side carrying no endpoint is a TELEDATA's destination, unsatisfied exactly when its port is occupied; the free position nearest that port then has a neighbour towards it that cannot be free without being nearer, so that edge is legal too. Algorithm 1 therefore needs no abandonment branch and no ready-or-preparable filter on selection. The argument does not bound preparation's *length*, so the implementation keeps a repeated-state check and an iteration budget; neither has been observed to fire.

E. Plan Scoring and the Selection Key

Every candidate is scored before any preparation SWAP for it exists, and the scored plans are ordered by one lexicographic key, smallest first:

$$\text{key}(\mathsf{p}) = \big(-\text{total},\ \text{prep},\ -f_{\text{dst}}\big), \tag{2}$$

each component evaluated at p, with a stable plan identifier resolving anything still tied. The order is the design, so we take the components in turn.

total is the estimated gate progress the plan buys, negated so that more progress sorts first. It decides almost every comparison — nine rankings in ten over the benchmark suites — and the two components after it only break ties.Preparation cost is not part of it, so a plan that happens to be cheap to prepare cannot displace one that buys more progress — the property that separates this scorer from DSABRE's, where staging cost is additive (Section VII).

prep is the readiness cost — how far the current layout is from satisfying the ready predicate of Section III-D, measured as intra-core distance from each endpoint to the port it must reach. It is zero exactly when the plan is ready, so it subsumes readiness itself: among plans of equal value, one that can fire now precedes one that cannot, and two unready plans are ordered by how much local work each still needs.

$-f_{\text{dst}}$ is the free capacity of the destination core, negated, and it does two different jobs. Between two TELEDATA plans it compares headroom, and the one leaving more room wins; it separates the winner from another TELEDATA in 265 rankings. Between a TELEDATA and a member of the TELEGATE family it compares nothing: only a TELEDATA consumes a destination slot, so the TELEGATE family scores zero here while any admissible TELEDATA scores $-f_{\text{dst}} \leq -f_{\min}$, and the TELEDATA sorts first whatever its destination holds. This is the component that carries the TELEDATA-first preference an explicit plan-kind rank used to provide, and it decides 627 rankings that way. In 230 of those the winning TELEDATA's destination sits at $f_{\min}$, so what the preference expresses is not that the plan leaves more room — it leaves less — but that relocating a qubit is worth more than discharging one straddling gate, because the pair it spends makes every later CX with a partner in that core local. It is computed alongside the score, never added to it.

A TELEGATE scores total=1 and a TELEGATEPACKET is only generated carrying at least two straddling gates, so $-\text{total}$ always separates those two primitives and only the TELEDATA-first half of a plan-kind ordering was ever reachable. Those are also the only totals the TELEGATE family takes, so a tie between primitives can fall only on a positive integer, and a TELEDATA reaches one only when its discounted gain is integral — in 1,426 of the 1,438 such ties it is zero, leaving both plans offering the same realised progress for the same pair. And since total is the sum of the realised and discounted gains, a trailing $-\text{lookahead}$ could separate two plans only where those two parts traded in exactly compensating amounts — once in 24,494 rankings over the benchmark suites, where it preferred the plan with the *smaller* realised gain. Removing all three leaves the emitted circuit bit-for-bit identical on every instance but two.

That leaves total, which the rest of this subsection computes, taking the three primitives in turn.

Two of them are counts of work the action performs outright. A **TELEGATE** scores 1: the one straddling gate it executes, with nothing left over. A **TELEGATEPACKET** scores the number of straddling gates in its collected run, also with nothing left over, since collection returns a committed executable sequence and every member of it executes once the cat resource is live (Section III-F).

A **TELEDATA** executes no gate at all. Its score has to estimate the gates its moving qubit q will meet once relocated, which needs a measure of how much a move brings a gate closer to being executable.

Gate progress. Write κ for the map sending a logical qubit to the core holding it under the current layout, and κ' for the same map after the move — identical to κ except that q now sits in c_{dst}. For a CX $g = (q_i, q_j)$,

$$\Delta(g) = d_{\text{C}}\big(\kappa(q_i), \kappa(q_j)\big) - d_{\text{C}}\big(\kappa'(q_i), \kappa'(q_j)\big) \tag{3}$$

is the core-graph distance the move removes from g. A TELEDATA moves one qubit one hop, so $\Delta(g) \in \{1, 0, -1\}$: 1 when the hop brings g's operands a core closer, whether or not that makes them local; 0 when it does not touch g; -1 when it pushes them apart.

Three sets of CX gates on q contribute, obtained by simulating the DAG frontier forward under κ':

- $D(\mathsf{p})$, the front CX on q — by the same one-gate-per-qubit fact, the plan's seed gate and nothing else, so $|D(\mathsf{p})| = 1$.
- $L(\mathsf{p})$, the CX gates on q reached by *local closure*. The simulator repeatedly retires every front gate executable under κ' — one-qubit gates, and CX gates κ' places in one core — advancing the front and exposing gates blocked behind them. Every CX on q appearing in any front along the way joins $L(\mathsf{p})$, and closure stops at the first front with nothing locally executable. These are current pressure rather than prediction: reaching them needs no gate outside κ''s reach.
- $A_k(\mathsf{p})$, the CX gates on q found by continuing past that stopping front, in simulated layer $k \le L_{\text{tp}}$, discounted by $w_k = \max(w_{\min}, w_0\gamma^{k-1})$ with w_0=0.9, γ=0.9, $w_{\min}$=0.5. These are prediction. The depth L_{tp} is counted in DAG layers; a companion cap N_{tp} on the gate count cannot bind while $N_{\text{tp}} \ge L_{\text{tp}}$, since a qubit contributes at most one gate per layer (Appendix H). L_{tp} is the one parameter PORTER does not fix by constant (Section IV-E1).

Only here does the score split, into a certain part and a discounted one: $\text{total} = \text{direct} + \text{lookahead}$, where

$$\begin{aligned} \text{direct}(\mathsf{p}) &= \sum_{g \in D(\mathsf{p}) \cup L(\mathsf{p})} \Delta(g), \\ \text{lookahead}(\mathsf{p}) &= \sum_{k \ge 1} \sum_{g \in A_k(\mathsf{p})} w_k\, \Delta(g). \end{aligned} \tag{4}$$

So $L(\mathsf{p})$ enters at full weight alongside $D(\mathsf{p})$, and only $A_k(\mathsf{p})$ is discounted.

F. Same-Control Packet Collection

Same-control CX gates are chained by DAG dependency edges, so they never appear in the front together: a packet cannot be read off the front and must be collected by forward simulation. From a seed adjacent-remote CX $g = (q_c, q_t)$ with target core c_t, PORTER simulates the DAG frontier forward for up to 120 layers, absorbing any front CX that shares control q_c and has its target in c_t (up to 8 gates), and any front 1Q gate whose qubit is already in c_t, since those execute freely while the cat resource is live. Collection stops at the first unproductive layer or at the size cap. The window counts simulated layers rather than collected CXs, so it is a cost guard rather than a tuning knob, and because the simulation reaches past the current front a packet may contain gates a front-only scan could not. One heuristic here is unablated: stopping at the first unproductive layer can miss a packet separated by harmless intervening work.

Collection only *discovers* the run; nothing is emitted while it simulates. The sequence is recorded on the plan and, if that plan is committed, emitted in exactly that order — one `cat_entangler`, consuming the packet's single EPR pair; one `cat_cx` per member, each target first brought beside the port by local SWAPs that may not touch either live port; and a closing `cat_disentangler` freeing both. Fixing the order at collection time is what lets the scorer count every member as work the packet will do: the members are DAG-chained, so that order respects their dependencies and none can become unexecutable once the resource is live.

G. Capacity Relief

Both tiers answer the same question — some core c is at or below $f_{\min}$ and a TELEDATA wants to enter it — by moving a qubit *out* of the congested region rather than into it. Neither picks a destination freely: a breadth-first search from c over the core graph finds the nearest *reservoir*, a core with at least $f_{\min}$ free positions, and the relief action moves one qubit from the penultimate core of that path into the reservoir at its end. Every logical qubit in the penultimate core is a candidate, scored and ordered by the plan machinery already described and grouped by hop count so shorter paths are tried first. Chaining beyond one hop is on by default, letting relief propagate when the immediate neighbour is itself full. What is

committed is a single hop, not the path: control returns to the top of Algorithm 1, which regenerates every plan, so the path chooses one eviction rather than binding the router to a route. A two-hop path frees the intermediate core, not the congested one, and only the next iteration can carry relief inwards. That granularity is what separates PORTER's relief from an atomic relay (Section VII) and what lets the opportunistic tier compete with ordinary work on the same scoring.

The two tiers differ only in when they fire. **Tail relief** fires reactively, when $\mathcal{P}$ is empty and the front is starved. **Opportunistic relief** fires pre-emptively, while a normal plan is still available, whenever a blocked plan out-scores it by more than m_{assist} — the judgement being that clearing the congestion now is worth more than taking the best currently legal action. The pre-emptive tier also searches less far: when its seed is blocked on an *adjacent* core it is capped at a single hop, so it fires only if some neighbour of the congested core is already a reservoir.

H. Progress and the Failure Model

Deterministic greedy routing on a capacity-constrained machine can stall, and PORTER layers four defences. It keeps capacity away from zero: a core with no free position is terminal, so a TeleData enters c only while $\text{free}(c) \geq f_{\min}$, leaving one position behind. Routing therefore cannot create a terminal core, and a supplied layout containing one is legalised first by evicting a qubit into the core with most room — the only input PORTER refuses is one where no core can receive that eviction. It detects starvation rather than thrashing, an empty $\mathcal{P}$ beside a non-empty $\mathcal{B}$ naming the cores responsible. It manufactures capacity, tail relief turning that signal into work (Section III-G). And it bounds every loop, hashing its decision state once per preparation round and raising if it repeats, under two budgets. Together these make every returned route correct and every run terminating, but not every run successful — the guarantee dSABRE has and PORTER does not (Section VII). A per-iteration cost analysis is in Appendix I.

IV. Experimental Methodology

A. Research Questions

The evaluation answers four questions, each with its own controls, and Section V is organised around them.

RQ1 End-to-end cost. Under each compiler's own recommended protocol, how does PORTER's EPR consumption and compile time compare with TeleSABRE's and dSABRE's?

RQ2 Routing quality with the layout controlled. Given the same, possibly externally supplied, initial layout, how does PORTER's routing loop compare with dSABRE's — first at an equal route count, then when each is allowed its own refinement schedule?

RQ3 Mechanism attribution. What does each of PORTER's mechanisms — capacity relief, the packet primitive, the TeleGate family, the initial layout and horizon selection — contribute on its own?

RQ4 Robustness. How do the answers change with circuit width, core count and core-graph shape?

Each results subsection opens with the question it answers, what is held fixed, and the metric; the interpretation of the answers is collected in Section VI, and the assumptions and limitations they rest on in Appendix K.

B. Terminology

Budget comparisons depend on using these terms consistently, so we fix them here and use no synonyms.

- A **route** (equivalently a *routing pass*) is one invocation of the routing loop of Section III on one circuit from one layout, producing one operation stream.
- A **candidate layout** is one initial mapping a protocol supplies to the router. A protocol's cost is its layout count times its routes per layout.
- A **protocol** is a layout source, a route budget and a selection rule; Table I names the ones used in the main results.
- A **candidate horizon** is one value of L_{tp} offered to horizon selection; selecting among them costs one route each.
- **Complete compile cost** is the wall-clock time of an entire protocol — layout generation plus every route over every candidate — not of the winning route alone.

For the machine and the router we use: *core*, *communication port*, and *reservoir* (a core holding at least $f_{\min}$ free positions); a front CX is *local*, *adjacent-remote* or *non-adjacent-remote*; a plan is *normal* or *blocked*, and *ready*, *preparable* or neither. A run that does not complete does so in one of four ways, distinguished throughout: a capacity *deadlock* leaves no legal plan and no relief path, and raises a diagnostic; a *livelock* makes local progress indefinitely while retired gates and EPR count stand still, and is stopped by the no-progress guard; a *timeout* exceeds the wall-clock cap; and an *infeasible input* is one the machine has no spare capacity for, refused before routing begins (Section III-H).

C. Setup

Implementation. PORTER is Python, released with the harness. It takes no seed: heuristic ties resolve through a fixed chain of deterministic keys and any residual tie through a constant-initialised generator, so one route on a fixed circuit and hardware emits a byte-identical stream across runs, horizon selection included: a speculative route is bounded by the DAG layers its lookahead walk may peel, at most seven times the count of the route the configured horizon produced, so the bound is a property of the route rather than of the machine. The horizon each run selected is recorded with its result. Every reported PORTER route is certified by a replay verifier written separately from the router, which re-derives gate coverage, dependency order and physical legality from the emitted stream alone and is itself tested by fault injection (Appendix D).

Architectures. We use the two grid-of-grids topologies dSABRE [11] evaluates on — architecture A, a 2×2 array of 3×3 cores (36 physical qubits), and architecture H, a 2×3 array of 4×4 cores (96 physical) — adding its 5×5-core

scalability series in Section V-D and two networks of IBM 27-qubit heavy-hex tiles in Section V-E. All are read from the baselines' own device files, so every compiler sees identical physical qubit ids.

Benchmarks. We use the same MQT-Bench [9] files DSABRE's evaluation uses, confirmed by exact CX-count match: a 25-qubit suite (AE, GHZ, Graphstate, QFT, QNN, Random) on architecture A, and a 64-qubit suite adding QPEexact, QAOA and Multiplier on architecture H. We add an 80-qubit suite of the same nine families; its AE and QNN instances are not produced by the stock generator and were reconstructed to match its construction.

Baselines. We compare against TELESABRE [10] and DSABRE [11], the latter in its default configuration — safe mode, which makes capacity a legality condition with a per-core reserve and recovers by relaying slack along a BFS path (Section VII).

Three decisions avoid disadvantaging DSABRE. Its recovery budgets derive from the architecture as its own drivers do, since a binding iteration budget would cut its heuristic search short and hand the remainder to its guaranteed drain at a worse EPR count. We do not charge it for pre-routing layout repair, which lands in its EPR counter although it adjusts the starting layout rather than routing anything; we net that out for both compilers. And we take its layout rule exactly as it ships (Section IV-D).

TELESABRE's stock C binary crashes deterministically on architecture A, in its initial-layout pass rather than in routing. All TELESABRE numbers use a minimally patched binary, released with ours, that falls back to the least-loaded core; it moves only qubits the stock binary leaves unplaced, and architecture-H results are unchanged. Every compiler invocation is capped at 900 s wall-clock, and a call exceeding it counts as a failure to complete.

D. Initial-Layout Protocols

We evaluate under three initial-layout protocols, designed to separate routing quality from initial-layout quality.

Own-protocol comparison. Each router is first evaluated under its own established best-practice protocol. For the two baselines that is best-of-three, matching the convention already used to compare TELESABRE and DSABRE to each other [11]: TELESABRE runs its built-in Hungarian initial-layout heuristic at config seeds 0–2, and DSABRE runs Qiskit `SabreLayout` on its corners-removed architecture graph at seeds 0–2, each layout routed by SABRE-style forward → backward → forward passes; both report the best EPR count. PORTER instead builds one deterministic temporal locality-greedy layout (Section III-A) and routes it at each of the four candidate lookahead horizons of Section IV-E1, keeping the best — four routing passes over one layout, against nine over three. It searches no layouts, so every number is reproducible from the circuit file alone.

DSABRE's SabreLayout runs on a coupling map with the most remote minimum-degree (corner) physical qubits of every core removed, the count per core fixed by its own fill-adaptive rule. Removing them from the map means no logical qubit is placed there, so they begin free and hold spare capacity for arriving teleports and evictions; they are not communication ports, which are the designated qubits at the link endpoints (Section II-A). We use the rule as it ships, which is why our DSABRE figures reproduce those it publishes for the architectures it reports.

Shared-layout comparison. To separate layout quality from routing quality, both DSABRE and PORTER are then run from an identical externally supplied mapping — TELESABRE's own initial layout, and separately `pytket-dqc`'s hypergraph partition (Appendix B) — so the only difference left is the routing loop. **Cross-layout comparison.** Finally each compiler routes the other's own layout, which closes the remaining cell of the design.

E. Operating Protocols

Section III defines one routing pass. We evaluate two protocols built on it. PORTER routes its own layout once at each of four candidate lookahead horizons — $L_{\mathrm{tp}} \in \{4, 8, 16, 128\}$, one fixed set applied to every circuit — and keeps the lowest-EPR result; PORTER+R refines backward from that selected route. Both are deterministic and both keep the best result by EPR, so neither can return a worse route than the pass it contains. They invoke the router repeatedly rather than changing what a pass does, which is why they are described here and not in Section III.

1) PORTER*: choosing the horizon per circuit:* The teleport-lookahead horizon L_{tp} is the one parameter of Section III-E for which no constant is best across the corpus. In an offline sweep over $\{4, 8, \ldots, 1024\}$ the best value spans the whole range — 80-qubit QNN wants 4, the 25-qubit suite 16, every 100-, 200- and 360-qubit instance 128 — and the gap between PORTER's configured L_{tp}=16 and the per-instance best is 17.2% geometric-mean EPR.

Interaction density sets the direction. Let ρ be the mean, over logical qubits appearing in at least one CX, of the distinct partners each shares a CX with anywhere in the circuit, divided by $n-1$: one scalar summarising the whole circuit, not a per-qubit or horizon-limited quantity. So $\rho \to 1$ is all-to-all and a linear CX chain gives $\rho \approx 2/(n-1)$. It is measured on the transpiled benchmark rather than the textbook algorithm, and the two can differ sharply: optimisation level 3 drops the controlled-phase rotations whose angle underflows its tolerance, which leaves QFT between 30 and 37 partners per qubit at every width above 25, so its ρ falls from 0.92 at 25 qubits to 0.10 at 360. A circuit family need not stay on one side of the threshold as it scales. Near ρ=1 a long horizon finds the moving qubit wanted in every core at once, the gain estimate degenerates into noise, and short horizons win; where ρ is small the signal is coherent and long horizons win. It is not a usable predictor alone, though. Its rank correlation with the per-instance best horizon is -0.765 (Spearman), strong but not decisive: in the band $0.42 \le \rho \le 0.53$ where most 64-qubit circuits sit, QAOA wants 8 and AE wants 128, and a threshold rule on ρ recovers only 8.0% of the available 17.2%. That one scalar cannot separate them is unsurprising — the horizon acts per moving qubit, while ρ averages over all of them — and a per-qubit or horizon-limited density is untried.

PORTER therefore does not use ρ at all. Every circuit is routed at the same four horizons, $L_{\mathrm{tp}} \in \{4, 8, 16, 128\}$, and the lowest-EPR route is kept — four routes over one layout. We prefer this to a rule keyed on ρ, or on any other circuit statistic, because such a rule is fitted on the same instances the evaluation reports and buys nothing: measured over all forty-eight instances the fixed set matches a tuned two-set rule on the main suite and is 0.2% better pooled, and under backward refinement it is identical on every one of the twenty-four main instances. That comparison was measured on the router as it stood before the capacity-relief revision of Section III-G, which changed the routes but not the horizons they are selected from; we report it as measured rather than re-running the grid. What the rule was buying was compile time, not EPR — it kept the long horizon away from near-all-to-all circuits, where it does not converge — and the fixed set pays that back at 1.6 times the compile time for one pass and 1.3 with refinement. Three values are not enough: the best three-value set, $\{4, 16, 128\}$, is 0.5% worse than the rule on the main suite and 0.2% worse pooled. Nor is this particular quartet distinguished; $\{4, 16, 32, 128\}$, $\{4, 16, 48, 128\}$ and $\{4, 16, 64, 128\}$ all land within a percent of it, and the last is slightly better pooled. We take $\{4, 8, 16, 128\}$ because it is the union of the two sets the retired rule chose between, so it introduces no horizon value the compiler did not already use. Two rules bound the cost: the configured horizon — L_{tp}=16, the default of Section III-E, which the set contains — is routed first, unguarded, and wins ties; each other candidate is bounded by the DAG layers its lookahead walk may peel, at most seven times what that first route peeled. Hence a conditional guarantee — *provided the configured-horizon route completes*, selection never returns a higher EPR count than that route alone.

The candidate set was fixed on the corpus it is evaluated on. The sweep and the four values were chosen by inspecting the 48 instances, with no held-out family, so evidence for the *size* of the gain is in-sample. What that risks is bounded, and less than a selection rule would risk: the set is a portfolio rather than a prediction, so a poor choice costs compile time and cannot worsen the selected route; the configured horizon is a member and wins ties, so no instance can do worse than routing once; and because nothing is conditioned on the circuit, there is no threshold to generalise. Several other four-value sets perform within a percent, which is the evidence that the result does not turn on these particular numbers. An out-of-sample validation remains to be done.

2) PORTER+*R: refining backward from that route:* DSABRE routes each candidate layout forward, backward and forward again, so comparing route budgets rather than layout counts alone needs a PORTER protocol that spends the same effort. PORTER+R is SABRE's bidirectional layout refinement [12] applied to whole routes. Write $R(C, \ell)$ for routing circuit C from layout ℓ, and ℓ'_i for the layout pass i ends on. Pass 1 computes $R(C, \ell_0)$ and ends on ℓ'_1; pass 2 computes $R(C^{\mathrm{rev}}, \ell'_1)$ on the reversed circuit, *discards its route* and keeps only ℓ'_2; pass 3 computes $R(C, \ell'_2)$. Reversing C reverses every dependency edge, so routing it from where the forward pass ended yields a layout adapted to the *start* of the forward circuit. Odd pass counts above three repeat the cycle; P=5 throughout, the best forward pass is kept, and nothing is random.

Pass 1 *is* PORTER: it runs the selection above, and the horizon it chooses is held for the remaining $P-1$ passes — re-selecting each pass would multiply the count by the number of candidates to re-decide a property refinement does not change. That is what makes PORTER+R contain PORTER, so with the best forward pass kept it can never report more EPR. Total routes are $|\mathcal{H}| + P - 1 = 8$, and the cost is wall-clock rather than passes: a deep horizon makes a later pass dear, and on one instance a pass is stopped by the per-pass guard.

One property matters for reading the results. Refinement consumes a post-routing layout, and routing destroys the per-core free-slot reservation ℓ_0 was built with (Section III-A), so an intermediate pass may start from a layout worse than ℓ_0 for PORTER's capacity machinery. Individual passes therefore regress on some instances; only the best-pass rule keeps that from surfacing.

F. Comparison Protocols and Budgets

Table I states, for every configuration that appears in the results, where its initial layouts come from, how many routing passes it spends, and what it selects on. We use the names in its first column throughout.

Two notions of a matched budget are in play and neither is universally fair, so we report against both. Fixing the *initial layout* routes both compilers from one identical mapping — the shared-layout rows — which controls the layout but not the effort spent on it: PORTER spends four routes on its candidate horizons to DSABRE's three passes. Equal *candidate-layout sets* instead hand PORTER the very layouts DSABRE generated for itself, so both choose their best from the same three starting points; that equalises the search over layouts rather than the routing spent on each. Wall-clock is not equalised anywhere, but every table reports complete compile time so a reader can renormalise. PORTER both spends more routes in the shared-layout rows and costs more per route (Section V-F), so neither comparison understates it.

G. Metrics and Aggregation

The primary metric is EPR pairs consumed: TELESABRE counts teledata plus telegate operations, DSABRE counts teledata, and PORTER counts teleports plus telegates plus telegate packets, one pair per packet, all three the same physical resource. Intra-core SWAP counts and compile time are secondary and are never traded against EPR by any selection rule. Aggregates are unweighted geometric means over circuits, so each circuit counts equally; percentage changes are $100(b/a - 1)$ against the stated baseline and ratios are b/a, both computed from those means rather than averaged per circuit. Failures are never imputed: a configuration that does not complete an instance is excluded from every mean it appears in, along with that instance's other columns where the comparison is paired, so a mean labelled (n) is over exactly the n circuits all its columns completed — which flatters a baseline that fails its hardest instances, and is why completion

TABLE I
EVERY CONFIGURATION IN SECTION V: LAYOUT SOURCE, LAYOUT COUNT $|\mathcal{L}|$, PASSES PER LAYOUT P, ROUTES, AND WHAT IS MINIMISED. P IS THE NUMBER OF ROUTES SPENT ON ONE LAYOUT: THE $|\mathcal{H}|{=}4$ CANDIDATE HORIZONS WHERE A HORIZON IS SELECTED AND NOTHING ELSE IS, THE REFINEMENT PASSES ON THE +R ROWS. ROUTES ARE $|\mathcal{L}|\cdot P$, PLUS THE $|\mathcal{H}|{-}1$ EXTRA HORIZON PROBES EACH +R SEED SPENDS ON ITS FIRST PASS. APPENDIX J GIVES THE ONE CONFIGURATION NOT LISTED.

Name	Layout source	$\lvert\mathcal{L}\rvert$	P	routes	selects on
Own protocol (RQ1, RQ4)					
TELESABRE	Hungarian, seeds 0–2	3	1	3	best EPR
DSABRE	SabreLayout, seeds 0–2	3	3	9	best EPR
PORTER	temporal locality-greedy	1	4	4	best EPR
PORTER+R	temporal locality-greedy	1	5	8	best forward pass
Shared layout (RQ2) — both compilers routed from one external layout					
DSABRE shared	TELESABRE's or pytket-dqc's	1	3	3	best pass
PORTER shared	the identical layout	1	4	4	best EPR
PORTER+R shared	the identical layout	1	5	8	best forward pass
Cross layout (RQ2) — each compiler on the other's own layout					
DSABRE on PORTER	PORTER's layout	1	3	3	best pass
PORTER on DSABRE	DSABRE's candidates	3	4	12	best EPR
Ablation (RQ3) — one fixed layout, one fixed horizon					
PORTER ablation	temporal locality-greedy	1	1	1	—

counts accompany every such mean. Compile times are wall-clock on one otherwise idle machine (Apple M2, macOS 26.5, CPython 3.13), measured once per configuration with no warm-up, so they are indicative and should be read as ratios between protocols rather than as absolute costs.

V. RESULTS

Each subsection answers one research question of Section IV-A. Protocol names and budgets are those of Table I; lower EPR is better throughout.

A. RQ1: End-to-End Cost Under Each Compiler's Own Protocol

Question: how do the three compilers compare when each uses its own recommended protocol? *Held fixed:* nothing — each compiler supplies its own layouts and pass schedule (Table I). *Metric:* geometric-mean EPR over jointly completed circuits, plus completion counts and complete compile time.

Table II summarises the three suites; Appendix H gives the per-circuit EPR and SWAP counts. Baseline counts come from each baseline's own harness, and on the eight 64-qubit circuits TELESABRE completes ours reproduces the counts DSABRE publishes for it [11] exactly.

Against TELESABRE the ordering is consistent across the suite. On block *(a)*, from one deterministic layout, PORTER lowers geometric-mean EPR by 59.7%, 44.9% and 68.8% at 25, 64 and 80 qubits, spending four routes against TELESABRE's three seeded runs, and every circuit TELESABRE completes improves, from −23.2% to −81.0%. Refinement adds to that at 25 and 64 qubits (63.5% and 46.9%) but little at 80 (70.3%), where the horizon rather than the extra routes does the work. TELESABRE completes seventeen instances, failing on 64q Random and six of the nine 80-qubit circuits; PORTER and DSABRE complete all twenty-four.

Against DSABRE the answer varies by suite, and block *(b)* is the basis since both complete everything. At four routes to DSABRE's nine, PORTER takes the geometric mean at 25 qubits (0.94×) and at 80 (0.93×) and trails at 64 (186.1 against 173.1, 1.07×), winning eleven instances, losing ten and tying three; pooled the two are level, 169.4 against 171.9 (0.99×). Refinement extends that at eight routes to 156.7 (0.91×) and narrows 64 qubits to 1.04×, so the one suite PORTER loses under its own protocol is also the one extra routes largely close. PORTER+R's largest saving is 80q Multiplier, 2,130 against 4,841; its worst loss 80q Random, 10,848 against 10,551.

B. RQ2: Routing Quality With the Layout Controlled

Question: with the initial layout held identical, does either routing loop use fewer EPR pairs? *Held fixed:* the layout, taken in turn from each of the three sources available. *Metric:* geometric-mean EPR over all twenty-four instances.

The three sources are those that place all twenty-four circuits. TELESABRE is not among them: it converges on seventeen and emits a layout only where it converges (Section V-A).

Changing the routing loop moves less than changing the layout. Across any row of Table III, from the same layout, the two land within 1.9% of one another on block *(a)* and within 6.5% on *(b)*, PORTER ahead in two rows of three in each block and DSABRE taking `pytket-dqc`'s partition. The layout is controlled but the budget is not: PORTER spends four routes on its candidate horizons to DSABRE's three passes, so a narrow PORTER lead here is not a like-for-like win. On the eighteen non-trivial instances the column spread is 9.2–14.5% against 0.9–6.1% across a row, so the initial layout is the larger source of variation for both compilers; pooled over all twenty-four the two are comparable, so that reading is a claim about the non-trivial circuits.

Which layout is best is not, and PORTER does not adopt the partition. Over all twenty-four instances `pytket-dqc`'s hypergraph partition is the lowest cell in

TABLE II

MAIN COMPARISON, OWN PROTOCOL: EACH COMPILER FROM ITS OWN INITIAL LAYOUT AND ROUTE BUDGET (TABLE I). BLOCK *(a)* AVERAGES THE CIRCUITS *all four* COMPLETE, THE ONLY BASIS ON WHICH TELESABRE CAN BE COMPARED; BLOCK *(b)* AVERAGES EVERY CIRCUIT AND DROPS IT. THE BLOCKS DIFFER ONLY IN WHICH CIRCUITS THEY AVERAGE. "DONE" IS CIRCUITS COMPLETED AND "W/L/T" THE PER-INSTANCE RECORD AGAINST DSABRE OVER ALL TWENTY-FOUR; t IS COMPLETE PROTOCOL TIME (GMEAN, SECONDS) OVER THE WHOLE SUITE. THE LAST PAIR IS THE PORTFOLIO†, A DERIVED QUANTITY RATHER THAN A FIFTH CONFIGURATION, SO IT IS LEFT UNBOLD. BOLD MARKS THE LOWEST EPR COUNT OF THE FOUR COMPILERS IN EACH ROW; TIMES ARE NEVER BOLD. LOWER IS BETTER; PER-CIRCUIT COUNTS IN APPENDIX H.

	TELESABRE		DSABRE		PORTER		PORTER+R		best of DSABRE with†	
Suite	EPR	t	EPR	t	EPR	t	EPR	t	PORTER	+R
(a) over the circuits *all four* complete										
25q (6)	106.6	0.51	45.8	0.23	43.0	0.21	**38.9**	0.36	41.9	38.9
64q (8)	284.5	11.89	**144.6**	1.82	156.8	2.24	151.1	3.93	140.5	139.6
80q (3)	121.0	5.11	44.9	3.59	37.8	5.00	**35.9**	9.35	33.8	33.8
(b) over *every* circuit in the suite; TELESABRE omitted										
25q (6)	—	—	45.8	0.23	43.0	0.21	**38.9**	0.36	41.9	38.9
64q (9)	—	—	**173.1**	1.82	186.1	2.24	179.6	3.93	168.8	167.4
80q (9)	—	—	412.1	3.59	384.7	5.00	**346.1**	9.35	344.7	333.4
all (24)	—	—	171.9	1.40	169.4	1.68	**156.7**	3.00	155.7	150.5
done	17 / 24		24 / 24		24 / 24		24 / 24		—	
W/L/T	—		—		11 / 10 / 3		14 / 8 / 2		—	

† **Portfolio.** For each instance, the lower of DSABRE's EPR count and the named PORTER configuration's, each compiler under its own protocol and its own initial layout: no layout is shared here, and neither router is given the other's mapping — for that control see Table III. The cell is the geometric mean of those per-instance minima over the row's circuits, which is why it can fall below *both* compilers' means: they lose on different circuits. At 64 qubits DSABRE is cheaper on six of the nine and PORTER on two, and PORTER's 1,076 against 1,285 on Multiplier is what carries the pair to 168.8, under DSABRE's own 173.1. Realising it costs both compile budgets, and no rule we tested predicts the per-instance winner in advance (Section VI).

TABLE III

LAYOUT AGAINST ROUTER: GEOMETRIC-MEAN EPR FOR EVERY COMBINATION OF INITIAL LAYOUT (ROWS) AND ROUTING LOOP (COLUMNS). PORTER SPENDS FOUR ROUTES PER LAYOUT TO DSABRE'S THREE PASSES, PORTER+R EIGHT OVER FIVE PASSES. BLOCK *(a)* IS OVER ALL TWENTY-FOUR INSTANCES; BLOCK *(b)* REPEATS IT WITHOUT GHZ AND GRAPHSTATE, WHOSE COUNTS ARE ONE TO TWO ORDERS OF MAGNITUDE BELOW THE REST AND SO DOMINATE AN UNWEIGHTED GEOMETRIC MEAN. THE BLOCKS DISAGREE IN EVERY COLUMN, WHICH IS THE TEXT'S SUBJECT. LOWER IS BETTER; BEST IN EACH COLUMN OF EACH BLOCK BOLD.

Initial layout	DSABRE	PORTER	PORTER+R
(a) all twenty-four instances			
DSABRE's `SabreLayout`	171.9	170.3	156.3
PORTER's temporal locality-greedy	171.2	169.4	156.7
`pytket-dqc`'s partition	**164.1**	**167.2**	**153.8**
(b) the eighteen excluding GHZ and Graphstate			
DSABRE's `SabreLayout`	**452.5**	448.7	426.2
PORTER's temporal locality-greedy	473.9	**445.2**	**422.7**
`pytket-dqc`'s partition	501.1	509.9	461.8

every column; drop GHZ and Graphstate and it is the highest in every column, raising the mean by 10.7% for DSABRE and 14.5% for PORTER. Those six rows carry the whole result, and the partition is the worse start on fourteen of the twenty-four instances for DSABRE and sixteen for PORTER, at a median ratio of 1.03 and 1.07. Its gain is real but confined to the two structurally simplest families, whose interaction graphs a partitioner cuts almost perfectly and whose absolute cost is least. It is also expensive: best-of-three KaHyPar seeds costs 0.6–627 s against 0.3–9.0 ms for PORTER's construction, on most circuits more than PORTER's whole four-route protocol, and it does not finish at the widths of Section V-D. Cost and confinement are why PORTER keeps its own layout and treats the partition as an opt-in candidate (Appendix C). Neither effect is specific to PORTER — the same partitions are the worst start for DSABRE's loop on the same eighteen instances — plausibly because `pytket-dqc` optimises e-bits on a network more permissive than the device [11].

Refinement pays from every starting layout, once the horizon is free to move. PORTER+R gains 8.2% over PORTER on DSABRE's layout (170.3 → 156.3), 8.0% on `pytket-dqc`'s (167.2 → 153.8) and 7.5% on its own (169.4 → 156.7), overtaking DSABRE's own protocol from all three at eight routes to its nine.

C. RQ3: Mechanism Ablation

Question: what does each mechanism contribute on its own? *Held fixed:* the initial layout, the lookahead horizon and every other parameter; exactly one mechanism is disabled per column. *Metric:* EPR and completion.

Table IV disables each mechanism on the 80-qubit suite, holding layout, horizon and every other parameter fixed; Appendix G gives the per-action counts. Every column is one forward pass at the configured horizon, so Full is that pass, not Table II's selected route. What each column asks is whether its mechanism earns its place.

Packets. Disabling TELEGATEPACKET makes every shared-control remote CX pay per gate. At 80 qubits that costs only 0.9% over the eight rows both columns complete, and the sign splits: AE pays +11%, QFT +6%, QNN +3% and QPEexact +2%, while QAOA is 5% and Multiplier 8% *cheaper* without packets — the two packet-heaviest rows in the table. The primitive still pays at the smaller widths (3.0% at 25 qubits, 7.4% at 64), so batching stops earning its keep where the front is widest and a packet's forward simulation displaces a better teleport plan. Read alone the column is near neutral; the next explains why.

TABLE IV
MECHANISM ABLATION, 80-QUBIT SUITE: PORTER EPR WITH ONE DESIGN DECISION DISABLED, FROM THE SAME LAYOUT AT THE SAME HORIZON. "NO PK" DISABLES PACKETS, "NO REL." BOTH RELIEF TIERS, "TELEDATA" THE TELEGATE FAMILY; "PK"/"TG" ARE THE FULL CONFIGURATION'S PACKET AND SINGLE-TELEGATE COUNTS. ALL NINE CIRCUITS ARE SHOWN; GHZ AND GRAPHSTATE COINCIDE IN EVERY COLUMN, NO MECHANISM BEING EXERCISED ON THEM. THE GMEAN LINES' n DIFFERS BECAUSE THE COLUMNS FAIL ON DIFFERENT CIRCUITS. †DOES NOT COMPLETE. ‡NEEDS THE EXTRA RELIEF TIER.

Circuit	Full	No pk	No rel.	TELEDATA	pk	tg
ae	331	367	†	342‡	54	1
ghz	7	7	7	7	0	0
graphstate	28	28	28	28	0	0
qft	318	337	†	†	78	0
qnn	1072	1099	†	1132‡	113	2
random	11245	†	†	†	537	3792
qpeexact	435	443	†	**347**‡	124	0
qaoa	1202	1146	1505	†	360	21
multiplier	3897	3584	3055	†	208	1739
gmean, n=8 of 9	**286.2**	288.9				
gmean, n=4 of 9	174.1		**173.3**			
gmean, n=5 of 9	124.8			**121.4**		

TELEDATA only. This column is two configurations. *Strict removal* of the TELEGATE family is the one-factor ablation, and it fails on all but two rows: the TELEGATE is the only remote action needing no free *data* slot, since its cat copy occupies the target-side communication qubit instead of displacing a logical qubit, so it alone is not gated on $\text{free}(c) \geq f_{\min}$. Without it a front of adjacent remote CXs whose destinations all sit at $f_{\min}$ has an empty plan set. The TELEGATE earns its place on *completion*, then, not on termination, which the guards of Section III-H give either way.

The vocabulary variant (‡) also feeds adjacent-blocked candidates to the relief BFS, recovering five of the seven; QFT, Random, QAOA and Multiplier still do not complete. That tier leaves the full configuration EPR-identical, so what it isolates is vocabulary and not relief — though it is no longer a strict ablation. On that basis teleport-only is 2.7% cheaper at 80 qubits, carried by one row — QPEexact at 347 against 435 — while it is *dearer* on AE (342 against 331) and QNN (1,132 against 1,072), so the ordering TELEDATA only < Full < No packets holds on one of the five rows where all three complete. Pooled over the nineteen it completes it is 3.5% dearer, on the same width trend as the packets column (+2.2%, +8.6%, −2.7%): teleports alone are competitive only at the widest circuits, and beat the full set outright nowhere. Section VI reads the residue as a currency mismatch in the plan comparator, not a verdict on packets. It also bears on DSABRE's open question: it chose teledata-only by measurement, leaving grouping untested, and grouping pays at the widths where its own suite sits.

Capacity relief. Disabling both tiers costs completion, not EPR pairs. Five of the nine circuits fail outright: with every destination below $f_{\min}$ and no way to drain qubits toward reservoirs, no legal remote plan remains. Where routing completes it is 0.5% *dearer* than doing without, and Multiplier is cheaper without it, so the opportunistic tier can overpay where congestion would have cleared itself: at this width relief earns its place on completion alone, though pooled over the nineteen that complete both ways it still saves 24.8%. With both tiers on no instance here reached the deadlock path of Section III-H; with both off, five did — a statement about these instances, not a guarantee.

TABLE V
SCALE-UP SUITE: QFT AND QPEEXACT AT 100, 200 AND 360 LOGICAL QUBITS, EPR PAIRS, EACH COMPILER UNDER ITS OWN LAYOUT AND PROTOCOL. ARCHITECTURES FOLLOW DSABRE'S SCALABILITY PROTOCOL (5×5 CORES, N_{phys} PHYSICAL QUBITS IN $K = 6 \to 12 \to 20$ CORES), READ FROM ITS OWN DEVICE FILES; OUR DSABRE COLUMN REPRODUCES ITS PUBLISHED SERIES [11] (201/246/766/896/1890/1938). "t" IS COMPLETE PROTOCOL TIME: NINE ROUTES FOR DSABRE, FOUR FOR PORTER, WHICH SELECTS L_{tp}=128 ON ALL SIX. PYTKET-DQC IS OMITTED — ITS PARTITIONER DOES NOT FINISH AT THESE WIDTHS. LOWER IS BETTER.

		architecture			DSABRE		PORTER	
Circuit	CX	n	N_{phys}	K	EPR	t	EPR	t
QFT	3,420	100	150	6	200	3.6	**198**	3.0
QPEexact	3,579	100	150	6	246	4.3	**200**	2.4
QFT	7,220	200	300	12	765	13.9	**564**	9.3
QPEexact	7,579	200	300	12	896	13.9	**528**	9.0
QFT	13,300	360	500	20	1890	53.9	**1074**	18.6
QPEexact	13,979	360	500	20	1938	83.1	**1093**	22.5
gmean (6)					705.7	15.4	**490.0**	**8.0**

D. RQ4: Scale-Up to 360 Logical Qubits

Question: do the RQ1 and RQ2 answers hold as circuit width and core count grow together? *Held fixed:* DSABRE's own scalability architectures and circuit files. *Metric:* geometric-mean EPR and complete compile time.

Every result so far is at 108 physical qubits or fewer. Table V takes the two circuit families that exist at 100, 200 and 360 logical qubits onto DSABRE's scalability series: 5×5 cores throughout, core count 6 → 12 → 20, so occupancy stays roughly constant while the core graph grows and inter-core traffic with it — DSABRE's own cost rises 200 → 765 → 1,890 on QFT.

PORTER uses fewer EPR pairs than DSABRE on all six instances, by a widening margin: 0.90× at 100 logical qubits, 0.66× at 200, 0.57× at 360, for 0.69× overall. It does so at 0.52× DSABRE's complete compile cost (8.0 s against 15.4 s), routing four times where DSABRE routes nine.

The horizon is what makes that difference, and the comparison is available directly. Routing the same six instances from the same layouts at the configured horizon alone — one route, no selection — gives a geometric mean of 1,026.5 pairs against DSABRE's 705.7, which is 1.45× and loses all six. Selection changes nothing else about the compiler and reaches 490.0, which is 0.69× and wins all six. Every instance selects L_{tp}=128, the longest candidate offered, and none selects the configured 16: at these widths a plan's value is realised further into the circuit than the default horizon can see.

E. RQ4: Heavy-Hex Cores and Non-Grid Core Graphs

Question: how much of the foregoing depends on square cores wired in a grid? *Held fixed:* the 64-qubit circuit suite

TABLE VI
THE NINE-CIRCUIT 64-QUBIT SUITE ON FOUR IBM 27-QUBIT HEAVY-HEX CORES (108 PHYSICAL), WIRED AS A *ring* (ALL CORES DEGREE 2) AND A *star* (CORE 0 A HUB EVERY ROUTE CROSSES). GEOMETRIC-MEAN EPR; BOTH COMPILERS COMPLETE ALL EIGHTEEN INSTANCES. THE LAST COLUMN IS THE FIXED-LAYOUT CONTROL — PORTER ROUTING DSABRE'S OWN SABRELAYOUT CANDIDATES, SO ONLY THE ROUTING LOOP DIFFERS. PER-CIRCUIT ROWS ARE IN APPENDIX H.

		PORTER	
Core graph	DSABRE	own layout	DSABRE's layout
ring (9)	85.2	86.1	**82.9**
star (9)	101.1	113.3	**97.2**

and the physical-qubit count; the core shape and core graph change. *Metric:* geometric-mean EPR and win/loss counts.

We adopt DSABRE's non-grid experiment [11]: the same nine-circuit 64-qubit suite on four IBM 27-qubit heavy-hex tiles. Two things change at once — the *core* becomes irregular (degrees over $\{1, 2, 3\}$, six degree-1 leaves, diameter 12 against a 4×4 grid's 6) and the *core graph* is either a ring, every core of degree 2, or a star with core 0 a hub. Neither compiler is specialised for a heavy-hex tile, and both are given the identical device.

The core graph matters more than the core shape. On the ring (Table VI) the two compilers are within 1% of one another, 86.1 against 85.2: doubling the intra-core diameter and replacing degree-4 interiors with degree-1 leaves disturbs neither. The star does. There PORTER trails by 12%, 113.3 against 101.1, winning two of nine. The core shape is the same in both rows; only the core graph differs.

The initial layout is the dominant observed source of the star gap. The last column of Table VI holds the routing loop against DSABRE's own `SabreLayout` candidates: routed from those, PORTER overtakes DSABRE on *both* core graphs — 82.9 against 85.2 on the ring, 97.1 against 101.1 on the star — and the reversal is larger on the topology it loses under its own layout. The cause is in the layout, and it is specific to the star: PORTER reserves $f_{\min}$ in every core, treating cores as interchangeable, which they are on a grid and a ring and are not where one core is a hub every route must cross. On star Multiplier that reservation opens with free counts $(2, 2, 13, 27)$, the hub pinned at the floor while a whole core sits empty.

This bounds the layout reading rather than extending it: up to 108 physical qubits, across two core shapes and three core graphs, the initial layout separates the two compilers and controlling it erases the difference — a bound that the scale-up series of Section V-D, where the same control does not help, keeps from being read as a general one.

F. The EPR/Time Operating Frontier

Table VII places PORTER's operating points (Section IV-E) against DSABRE, pooled over all twenty-four instances. Each is the previous one plus a search: PORTER selects the horizon and PORTER+R refines backward from the route it selects. Because each schedule contains the one before it and keeps the best result by EPR, the column is monotone by construction, and neither can report more EPR than routing once at the configured horizon.

TABLE VII
PROTOCOL TRADE-OFF OVER ALL 24 INSTANCES. "ROUTES" COUNTS ROUTING PASSES; TIMES ARE COMPLETE COMPILE COSTS (GMEAN PER INSTANCE). A THIRD SCHEDULE, WHICH ALSO SEARCHES THE LAYOUT DISCOUNT γ, IS REPORTED IN APPENDIX J.

Protocol	routes	EPR	vs. DSABRE	time (s)	vs. DSABRE
DSABRE	9	171.9	1.00	1.40	1.00
PORTER	4	169.4	0.99	1.68	1.20
PORTER+R	8	156.7	0.91	3.00	2.15

The trade is time against EPR pairs. PORTER costs $0.99\times$ DSABRE's EPR at $1.20\times$ its compile time; refinement extends the EPR lead to $0.91\times$ for $2.15\times$ the time, on eight routes against DSABRE's nine.

All times are *complete* compile costs — layout generation plus every route over every candidate, not the winning route alone. PORTER routes fewer times than DSABRE at every operating point, four against nine and eight against nine, and is still the slower of the two: its routes are the dearer ones, because each enumerates whole plans, forward- simulates packets and searches two tiers of capacity relief. Compile time is therefore not an argument for PORTER on these suites — we report it because the EPR figures are only meaningful beside the budget that produced them. It does become one at scale, where the same four routes run in $0.50\times$ DSABRE's complete cost (Section V-D).

VI. DISCUSSION AND FUTURE WORK

The two compilers are complementary, and a portfolio beats either. PORTER wins ten of the twenty-four instances against DSABRE and loses twelve, PORTER+R eleven and eleven (Table II), so neither dominates instance by instance even though their pooled means are level (169.4 against 171.9). Taking the lower EPR count of the two per instance gives 155.7, a 9.4% reduction on the better of them, and 150.5 with PORTER+R in place of PORTER, 12.5% below DSABRE. The gain is largest where the two disagree most — 16.4% at 80 qubits against 2.5% at 64 — and it is not an artefact of the two cheapest families: over the eighteen instances excluding GHZ and Graphstate it is still 8.0% and 8.9%.

That gap is what the pooled figure conceals, and it is the clearest evidence that the two designs are not interchangeable: a compiler that merely re-derived DSABRE's decisions by another route would leave a portfolio with nothing to choose between. A 9–12% gain instead means each compiler reaches circuits the other does not, and by wide margins where it does — 80-qubit Multiplier at 2,874 pairs against 4,841, 80-qubit QPEexact at 276 against 347, with DSABRE ahead elsewhere. Which of the differences between them produces any particular win is not something this measurement can say: the two differ at once in action vocabulary, candidate granularity, where preparation cost enters, commitment, initial layout, horizon policy and route budget, and the portfolio ranks outputs rather than separating causes. It is a property of the

pair rather than of either compiler: it costs both routes (2.3× DSABRE's compile time, thirteen routes to nine), and no rule we tested predicts the winner in advance.

Why TELEDATA-only can beat the full action set. Packets beat the single telegates they degrade into, yet a vocabulary with gate teleportation removed altogether — TELEDATA and nothing else — is 19.8% cheaper than the full action set over the 80-qubit circuits it completes, and cheaper on every one of them but QAOA, which it loses by 0.6%. The cause is a currency mismatch in the scorer: a packet's gain counts remote CX gates resolved immediately, a TELEDATA plan's a sum of core-distance reductions discounted over a bounded horizon. Nothing establishes the two are commensurate, and where a teleport repays past the horizon the comparator cannot see it. That is evidence the mismatch costs pairs at scale, not that spreading one pair over a run of gates is a bad idea.

A. Future Work

On the compiler itself. Four openings, the first a completion guarantee to match DSABRE's. Adopting its recovery wholesale is the obvious route, and the capacity half is nearly in place — both treat free slots as a legality condition with the same floor of two, and both relieve by breadth-first search to a reservoir (Section VII). What DSABRE adds is atomicity: it relays the chain in one all-or-nothing transaction, where PORTER re-plans each hop through the ordinary plan machinery, which is what lets the opportunistic tier compete with normal work at all (Section III-G). Nor would it buy what is missing. PORTER never reaches the capacity deadlock that proof rules out, and the one failure we observe is a local-SWAP livelock — heavy-hex star Multiplier at a fixed horizon — with capacity healthy throughout. The guarantee needs a progress argument for the intra-core stage, not a stronger capacity transaction. One configuration is worth trying first, because two of our results point at it: DSABRE's proof is written for a teleport-only router, and a teleport-only vocabulary is the cheapest we measured at 80 qubits, so a PORTER restricted to TELEDATA is both where the guarantee is likeliest to transfer and where the pairs are — though not for free, since that configuration completes twenty-one of the twenty-four instances here.

The other three: a horizon that scales with the core count K — L_{tp} is an absolute gate count, so a plan at twenty cores is scored over proportionally less of the circuit than at four, and every scale-up instance picks the longest candidate offered; a circuit statistic that predicts the horizon well enough to replace the portfolio with a single route, since ρ averages over every qubit and the whole circuit where the horizon acts on one moving qubit over a bounded depth — a per-qubit or horizon-limited density is the obvious candidate and is untried, and without one the horizon stays a portfolio whose candidate set was fixed on the corpus it is measured on (Section IV-E1); and a comparator that weighs a resolved gate against a latent distance reduction on one scale, which the vocabulary result indicates the present one does not. The mismatch is confined to total itself: the selection key of Section III-E no longer carries a separate discounted term, so a scale on which a resolved gate and a latent distance reduction are commensurable would have to come from the summation, not from a tie-break.

Beyond EPR count. The trace PORTER emits is unscheduled, so a lower pair count does not by itself imply a shorter makespan or a higher-fidelity result. A scheduling pass should batch inter-core actions whose links, ports and preparation SWAPs do not conflict, and account for entanglement-generation rate, storage lifetime and gate error [13], [14]; the objective should stay multi-objective, since pair count, depth and fidelity trade off in hardware-dependent ways. Lifetime bears on PORTER in particular: a packet holds one cat resource open across a whole run, so a lifetime bound caps packet length in a way our cost model does not, and the vocabulary result above would have to be re-read under it.

Richer interconnects. Every device evaluated here gives adjacent cores a single link and each core one to four ports. Widening that is the free variable: a second link turns an occupied port from an obstacle into an alternative, relieving preparation cost first, and it interacts with the capacity discipline, since more ports mean more ways to spend and to recover a free slot. Where the return on added links falls off against their hardware cost is a question for interconnect co-design.

VII. RELATED WORK

TELESABRE [10] is the published state of the art for this cost model, supporting teledata and gate teleportation on SABRE's extended-set scoring. The two differ chiefly in decision structure. TELESABRE admits an inter-core candidate only when it is already executable — operand adjacent to a free communication port, a spare slot at the destination — and scores it by the layout energy it leaves behind, a contracted-communication-graph Dijkstra distance over the front and extended sets. Staging is therefore carried by ordinary SWAPs, pooled with those candidates and scored the same way, so each iteration's minimum may be a SWAP that shortens a route or a remote action that resolves a gate. PORTER keeps local work strictly ahead of inter-core work, scores whole plans independently of preparation cost, generates preparation SWAPs only for the plan it commits to, adds the packet primitive, and treats capacity as a legality condition with its own relief search rather than as TELESABRE's safety-valve cap.

DSABRE [11] attacks the same problem and is a close relative. Both DSABRE and PORTER rank moves by SABRE's front-plus-lookahead Δ score, both are strictly locality-first, scoring an inter-core candidate only once no front gate is intra-core, both promote capacity from a scoring penalty to a legality condition with the same floor of two free slots, and both manufacture capacity by relaying a free slot from a reservoir along a BFS path rather than evicting arbitrarily. Four differences remain: PORTER's action vocabulary is larger; its unit of commitment is a whole plan rather than one hop; preparation cost enters as a tiebreaker rather than additively; and its initial layout is one deterministic construction rather than a best-of-N search. A fifth is a gap rather than a design choice, and it runs DSABRE's way: its safe mode carries

a termination-with-success proof under stated architectural hypotheses, so it guarantees *completion*, where PORTER guarantees only correctness and termination by construction (Section III-H) and has completion empirically, no instance being left unrouted under the selected-horizon protocol of Section IV-E1.

Other approaches. Partition-first compilers assign qubits to cores by hypergraph or community partitioning [15]–[17] and route the residue; `pytket-dqc` [16] is the mature representative, compared against in Appendix B under the caveat that its e-bit unit assumes long-lived entanglement and different communication memory. AutoComm [8] identifies gate sequences that can share one entangled resource, which PORTER's packet applies inside the routing loop rather than as a pre-pass (Section II-C). SABRE [12] supplies the scoring skeleton all three inherit.

Table VIII places these systems on the axes that matter here.

VIII. Conclusion

PORTER is a deterministic DQC compiler built on a plan-oriented remote stage: it scores a complete inter-core action on the gate progress it buys, before any preparation SWAP for it exists, and commits before preparing. Around that it places a capacity discipline with two relief tiers, a deterministic initial layout, and a teleport-lookahead horizon chosen per circuit from a fixed four-value portfolio.

Three conclusions are supported. First, on the evaluated MQT-Bench suites PORTER uses substantially fewer ($>40\%$) EPR pairs than published TeleSABRE and completes all twenty-four instances, including the seven on which TeleSABRE does not converge. Second, against dSABRE it is competitive on those same suites, $0.99\times$ its pooled EPR at $1.20\times$ its compile time under each compiler's own protocol, and its advantage grows with the machine, reaching $0.69\times$ on the 100–360-qubit scale-up series. Third, controlled experiments locate where these results come from rather than leaving them to the system as a whole, each by its own kind of evidence: an ablation shows capacity relief is what lets a deterministic-greedy design complete in practice; holding the initial layout fixed shows it, not the routing loop, accounts for most of the spread between the two compilers on those suites; and routing the scale-up series at a fixed horizon instead of a selected one turns $0.69\times$ into $1.45\times$, which is what carries PORTER past dSABRE as the machine grows. The two compilers also prove complementary rather than ranked: neither wins more than eleven of the twenty-four instances, and taking the better of the two per instance reaches 12.5% below dSABRE alone (Section VI).

TABLE VIII
Design axes of the compilers discussed here. "Commitment" is the unit a scoring decision is taken over. `pytket-dqc` is partition-first and does not route with these primitives, so several axes do not apply.

	TeleSABRE	dSABRE	pytket-dqc	PORTER
Paradigm	online	online	partition-first	online
Primitives	data, gate	data	(e-bit)	data, gate, packet
Local/remote	one pool	staged	—	staged
Commitment	one move	one hop	whole plan	whole plan
Prep. cost	in score	in score	—	tie-break
Layout	Hungarian, 3 seeds	SabreLayout, 3 seeds	hypergraph partition	one deterministic
Capacity	cap	legality + relay	partition	legality + relief
Termination	—	with-success proof	—	correct, terminates
Metric	EPR	EPR	e-bits	EPR

Acknowledgments

This work was partially supported by the Natural Science Foundation of China (Grant No. 12471437). The authors used Anthropic's Claude (via Claude Code) to assist with the PORTER implementation, the replay verifier and comparison harness, and the drafting of this manuscript. All AI-generated content was reviewed and verified by the authors, who take full responsibility for the correctness and originality of the work.

References

[1] D. Cuomo, M. Caleffi, and A. S. Cacciapuoti, "Towards a distributed quantum computing ecosystem," *IET Quantum Communication*, vol. 1, no. 1, pp. 3–8, 2020.

[2] P. Escofet, A. Ovide, M. Bandic, L. Prielinger, C. G. Almudever, S. Feld, E. Alarcón, and S. Abadal, "Revisiting the mapping of quantum circuits: Entering the multi-core era," *ACM Transactions on Quantum Computing*, 2024, arXiv:2403.17205.

[3] L. J. Stephenson, D. P. Nadlinger, B. C. Nichol, S. An, P. Drmota, T. G. Ballance, K. Thirumalai, J. F. Goodwin, D. M. Lucas, and C. J. Ballance, "High-rate, high-fidelity entanglement of qubits across an elementary quantum network," *Physical Review Letters*, vol. 124, no. 11, p. 110501, 2020.

[4] S. Daiss, S. Langenfeld, S. Welte, E. Distante, P. Thomas, L. Hartung, O. Morin, and G. Rempe, "A quantum-logic gate between distant quantum-network modules," *Science*, vol. 371, no. 6529, pp. 614–617, 2021.

[5] C. H. Bennett, G. Brassard, C. Crépeau, R. Jozsa, A. Peres, and W. K. Wootters, "Teleporting an unknown quantum state via dual classical and Einstein-Podolsky-Rosen channels," *Physical Review Letters*, vol. 70, no. 13, pp. 1895–1899, 1993.

[6] J. Eisert, K. Jacobs, P. Papadopoulos, and M. B. Plenio, "Optimal local implementation of nonlocal quantum gates," *Physical Review A*, vol. 62, no. 5, p. 052317, 2000.

[7] A. Yimsiriwattana and S. J. Lomonaco Jr., "Generalized GHZ states and distributed quantum computing," *arXiv preprint quant-ph/0402148*, 2004.

[8] A. Wu, H. Zhang, G. Li, A. Shabani, Y. Xie, and Y. Ding, "AutoComm: A framework for enabling efficient communication in distributed quantum programs," in *Proceedings of the 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, arXiv:2207.11674.

[9] N. Quetschlich, L. Burgholzer, and R. Wille, "MQT Bench: Benchmarking software and design automation tools for quantum computing," *Quantum*, vol. 7, p. 1062, 2023.

[10] E. Russo, E. Vinciguerra, M. Palesi, D. Patti, G. Ascia, and V. Catania, "TeleSABRE: Heuristic layout synthesis in multi-core quantum systems with teleport interconnect," in *Proceedings of the IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2025, pp. 749–758, arXiv:2505.08928. Code: https://github.com/Haimrich/telesabre.

[11] S. Li, "dSABRE: A SABRE-style router for multi-core distributed quantum computers," 2026, manuscript, Centre for Quantum Software and Information, University of Technology Sydney. Code: https://github.com/ebony72/dsabre.

[12] G. Li, Y. Ding, and Y. Xie, "Tackling the qubit mapping problem for NISQ-era quantum devices," in *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019, pp. 1001–1014.

[13] J. M. Baker, C. Duckering, A. Hoover, and F. T. Chong, "Time-sliced quantum circuit partitioning for modular architectures," in *Proceedings of the 17th ACM International Conference on Computing Frontiers (CF)*, 2020, pp. 98–107.

[14] D. Ferrari, A. S. Cacciapuoti, M. Amoretti, and M. Caleffi, "Compiler design for distributed quantum computing," *IEEE Transactions on Quantum Engineering*, vol. 2, pp. 1–20, 2021.

[15] P. Escofet, M. Bandic, C. G. Almudever, and S. Feld, "HQA: A hungarian-based qubit assignment for multi-core quantum computers," in *Proceedings of the IEEE Quantum Software Conference*, 2023.
[16] P. Andres-Martinez, D. Mills, T. Forrer, and L. Henaut, "CQCL/pytket-dqc," https://github.com/CQCL/pytket-dqc, Jun. 2024.
[17] O. Daei, K. Navi, and M. Zomorodi-Moghadam, "Optimized quantum circuit partitioning," in *International Journal of Theoretical Physics*, vol. 59, 2020, pp. 3804–3820.



Sanjiang Li received his B.Sc. in mathematics from Shaanxi Normal University in 1996 and his Ph.D. in mathematics from Sichuan University in 2001. He is currently a professor at the Centre for Quantum Software and Information at the University of Technology Sydney (UTS). Prior to joining UTS, he worked in the Department of Computer Science and Technology at Tsinghua University from 2001 to 2008. His primary research interests include knowledge representation, artificial intelligence, and quantum circuit compilation.